AI-Powered Handwriting Generation System

Generating Personalized Handwritten Assignments
from Character-Level Deep Learning

Project Phase 1 Report

Contents

1. Introduction

1.1 Motivation

Handwritten documents are a fundamental part of academic life. Students are regularly required to submit handwritten assignments, notes, and reports. However, producing large volumes of handwritten content is time-consuming and physically demanding. A system that learns a user's personal handwriting style from a small set of character samples and automatically generates realistic handwritten assignment pages would offer significant value in terms of time saving and accessibility.

Rather than attempting complex sentence-level style transfer, our approach focuses on **character-level learning** — collecting individual handwritten character samples from the user and using a trained neural network to reproduce any typed text in that user's unique handwriting style. This makes the problem tractable, the dataset easy to collect, and the output consistent and high quality.

Challenges:

- Accurately segmenting and normalizing individual handwritten characters from a photographed sheet.
- Training a neural network to learn the unique visual features of each character (slant, stroke weight, curvature) per user.
- Rendering individual generated characters into natural-looking full lines and pages with realistic spacing and baseline variation.
- Collecting a sufficiently diverse character dataset from multiple writers to improve model robustness.
- Keeping the system simple and implementable within university project constraints.

Applications:

- Automated generation of personalized handwritten academic assignments.
- Accessibility tools for individuals with physical writing difficulties.
- Forensic handwriting analysis and writer verification research.
- Educational tools for practising and improving handwriting.
- Digital document signing in one's own handwriting style.

1.2 Problem Statement

Given a sheet of individually handwritten characters (A–Z, a–z, 0–9) photographed by the user, design and train a character-level Convolutional Neural Network that learns the user's handwriting style and generates full-page handwritten assignment documents that visually replicate the user's own writing.

2. Background / Literature Review

The following four works are most relevant to our proposed system.

2.1 ScrabbleGAN: Semi-Supervised Varying Length Handwritten Text Generation (Fischer et al., 2020)

ScrabbleGAN presents a fully convolutional GAN capable of generating handwritten text of arbitrary length. A character-level generator is combined with a CTC-based text recognizer to ensure generated text is both realistic and legible. Style is controlled via a latent style vector, enabling diverse handwriting appearances across writers.

Relevance to our work: The character-level generation idea directly aligns with our approach of building and stitching individual character images to form words and lines.

2.2 GANwriting: Content-Conditioned Generation of Styled Handwritten Word Images (Kang et al., 2020)

GANwriting generates handwritten word images conditioned on both the text content and a specific writer's style code. The style code is extracted from as few as 15 reference samples per writer. This few-shot capability is significant because our users will also provide only a limited number of character samples.

Relevance to our work: Confirms that a small per-user sample set is sufficient to learn a meaningful handwriting style embedding.

2.3 Handwriting Transformers (Bhunia et al., 2021)

This paper introduces a transformer-based architecture where separate encoders handle content (text) and style (reference images), fused via cross-attention during generation. The model outperforms GAN-based methods on the IAM benchmark in both style consistency and character legibility.

Relevance to our work: Motivates the use of embedding-based style encoding, which we incorporate in our CNN-based style extractor.

2.4 Generating Sequences With Recurrent Neural Networks (Graves, 2013)

This foundational work demonstrated that LSTM networks can generate realistic cursive handwriting by predicting pen stroke sequences conditioned on input text. It established the theoretical groundwork for all subsequent handwriting generation research.

Relevance to our work: Provides the conceptual basis for sequence-aware character placement and spacing in our page rendering stage.

(Every Reference is checked and copied from an LLM)

3. Dataset Details

As per project requirements, we do **not** use any preprocessed or pre-cleaned dataset. All data is collected raw and all preprocessing is performed entirely by us.

3.1 Data Collection Strategy

Method A — Self-Collected Character Dataset (Primary):

Each group member writes every character (A–Z uppercase, a–z lowercase, 0–9 digits) multiple times on plain white A4 paper using a standard pen. This gives us:

- 62 unique character classes (26 + 26 + 10)
- Each character written 5 times per person = 310 samples per person
- 4 group members $\times$ 310 = **1,240 raw character samples**

  (Maybe from more persons as well)
- Pages are photographed using a smartphone (minimum 12 MP) under natural daylight for consistent lighting
- Saved as PNG at minimum 300 DPI

(Maybe some automated tool will be used that generates digital handwriting samples and gives us the same result)

Method B — NIST Special Database 19 (Secondary):

The NIST SD19 database provides raw, unprocessed handwritten character images from hundreds of writers covering A–Z, a–z, and 0–9. We will use the original raw binary image files and perform all segmentation and preprocessing ourselves.

3.2 Preprocessing Pipeline (Performed by Us)

#	Step	Description
1	Binarization	Convert raw colour photos to black-and-white using adaptive Otsu thresholding in OpenCV.
2	Skew Correction	Detect and correct page tilt using the Hough line transform so characters are upright.
3	Character Segmentation	Isolate each individual character cell from the written grid using connected component analysis.
4	Normalization	Resize every character image to 64×64 pixels and normalize pixel values to $[0,\ 1]$.
5	Writer Labelling	Tag each sample with a unique writer ID to enable per-user style learning.
6	Data Augmentation	Apply small random rotations ($\pm 5^{\circ}$), slight elastic distortions, and brightness variation to increase dataset size.

(Using an agile method — may vary while developing)

4. Proposed Methodology

4.1 Why CNN and Not GAN?

For our character-based approach, a **CNN (Convolutional Neural Network) Encoder–Decoder** is the most suitable architecture for the following reasons:

- We already have real character images from the user, so we do not need a GAN to *invent* new styles — we only need to *learn and reproduce* an existing one.
- A CNN Encoder–Decoder is simpler to train, easier to debug, and produces stable, consistent output without the training instability problems common in GANs.
- GANs are more appropriate when generating diverse unseen styles from noise, which is not our use case.
- The CNN approach is well within university-level computational resources and is fully implementable from scratch in PyTorch.

4.2 System Overview

Our system takes two inputs: (1) the user's handwritten character sheet (photographed), and (2) any typed assignment text. It produces a full-page handwritten PDF that looks like the user wrote it by hand.

The complete pipeline is:

Character Collection → Preprocessing → CNN Training → Character Generation → Page Rendering (Output)

Character Sheet Photo ⟶ Preprocessing ⟶ CNN Style Encoder ⟶ Character Generator ⟶ Page Renderer ⟶ Handwritten PDF

Figure 1: End-to-End System Pipeline

4.3 Stage-by-Stage Description

Stage 1 — Character Collection: The user fills a printed template sheet with all 62 characters written in their natural handwriting. The sheet is photographed and uploaded. No sentence-level writing is required from the user.

Stage 2 — Preprocessing: The uploaded photo is processed through the 6-step pipeline in Section 3.2, producing 62 clean, normalized, labelled character images for that user.

Stage 3 — CNN Training (Core Neural Network): We build and train a **CNN Encoder–Decoder** from scratch in PyTorch. The network learns the visual style of each character from the user's samples. The architecture is:

- **Encoder CNN:** 4 convolutional layers with ReLU activation and max-pooling, extracting a compact style feature vector per character.
- **Decoder / Generator:** 4 transposed convolutional layers that reconstruct a clean handwritten character image from the style feature vector and a one-hot character label input.
- **Loss Function:** Mean Squared Error (MSE) loss between the generated character image and the ground truth, ensuring the network accurately reproduces the user's handwriting style.
- **Training:** Trained on the collected character dataset with an 80/20 train-validation split.

Stage 4 — Character Generation (Output): When given any typed text, the system passes each character through the trained Decoder with the user's style vector, producing a handwritten image of that character in the user's style.

Stage 5 — Page Rendering (Post-Processing): Individual generated character images are placed side-by-side to form words, words are arranged into lines with natural spacing, and lines are stacked onto a simulated A4 ruled page with margins. The final output is exported as a high-resolution PDF using PIL and ReportLab.

4.4 Neural Network Architecture Summary

Component	Type	Layers	Output
Style Encoder	CNN	4 Conv + MaxPool	Style feature vector
Generator	Transposed CNN	4 ConvTranspose	64×64 char image
Page Renderer	Post-processing	OpenCV + PIL	A4 PDF page

4.5 Tools and Technologies

- **Deep Learning Framework:** PyTorch
- **Image Processing:** OpenCV, PIL (Pillow)
- **Page Export:** ReportLab / PIL
- **Dataset:** Self-collected character sheets + Raw NIST SD19
- **Programming Language:** Python 3.x
- **Development Environment:** Jupyter Notebook / VS Code

References

[1] T. Fischer, C. Koniusz, and S. Gould, "ScrabbleGAN: Semi-Supervised Varying Length Handwritten Text Generation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 3508–3517, 2020.

[2] J. Kang, J. I. Toledo, M. Rusinol, D. Karatzas, and A. Gordo, "GANwriting: Content-Conditioned Generation of Styled Handwritten Word Images," in *Proc. European Conf. Computer Vision (ECCV)*, pp. 273–289, 2020.

[3] A. K. Bhunia, S. Khan, H. Cholakkal, R. M. Anwer, J. Laaksonen, M. Shah, and F. S. Khan, "Handwriting Transformers," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, pp. 683–693, 2021.

[4] A. Graves, "Generating Sequences With Recurrent Neural Networks," *arXiv preprint arXiv:1308.0850*, 2013.

[5] P. J. Grother, "NIST Special Database 19: Handprinted Forms and Characters Database," National Institute of Standards and Technology, 1995.

(Every Reference is checked and copied from an LLM)